

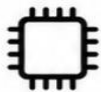
RISCForge

RTL-to-GDSII ASIC Implementation of a
5-Stage Pipelined RV32I RISC-V Processor
with Adaptive Branch Prediction,
Data Cache & Physical Memory Protection

INDEPENDENT PROJECT

Prepared by

Monika P



PROJECT DOMAIN

VLSI Design | Digital IC Design | ASIC Physical Design



KEY INNOVATION

Enhanced RV32I Processor with three architectural extensions:

- Adaptive Branch Predictor
- Direct-Mapped Data Cache
- Physical Memory Protection (PMP)



TECHNOLOGIES

Verilog HDL | SystemVerilog (UVM)



TOOLS

Visual Studio Code • WSL2 (Ubuntu) • Icarus Verilog • GTKWave
Yosys • OpenSTA • OpenLane2 • OpenROAD • Magic
Netgen • KLayout • Sky130 PDK • EDA Playground (Riviera-PRO)



YEAR
2026



GitHub: github.com/Monika-25-web



LinkedIn: linkedin.com/in/monika-p-b5ba5b400

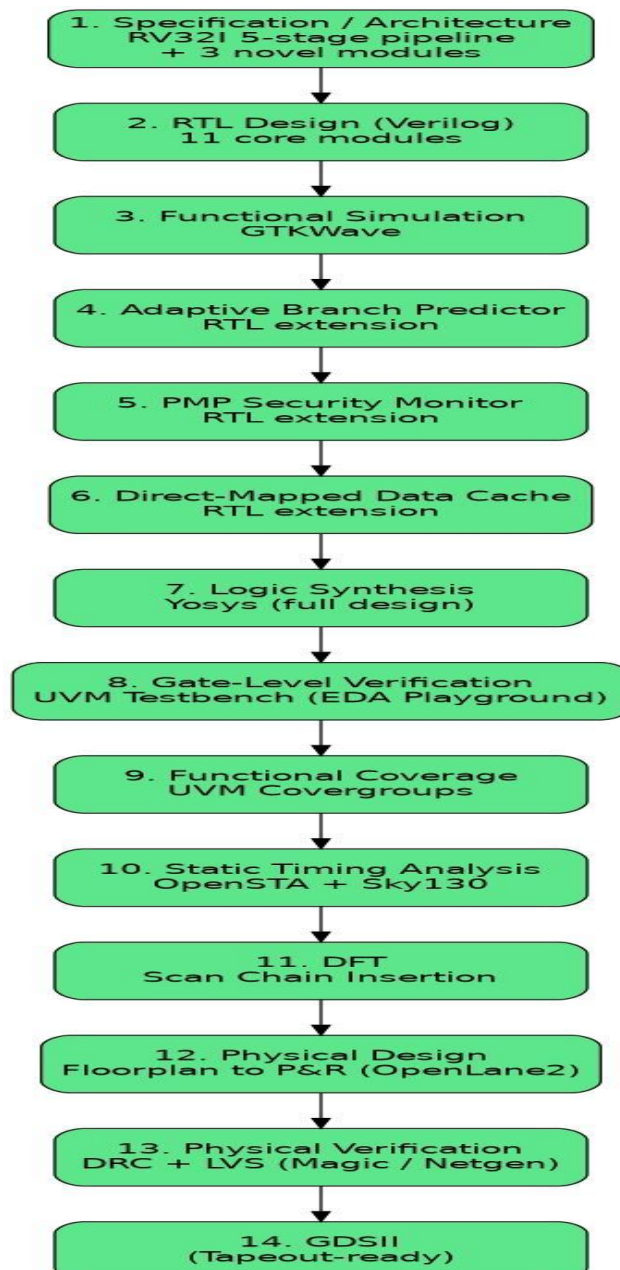


Email: monikapgowda11@gmail.com

Project Overview

This report documents the complete design flow of a 5-stage pipelined RV32I RISC-V processor, from architectural specification through RTL design, functional and gate-level verification, static timing analysis, design-for-test, and physical design, ending in a tapeout-ready GDSII file. The design includes three original architectural extensions: an adaptive branch predictor, a Physical Memory Protection (PMP) security monitor, and a direct-mapped data cache.

Full Project Flow



1. Specification / Architecture

Objective: Define the processor's target architecture and fix the project scope before any code is written.

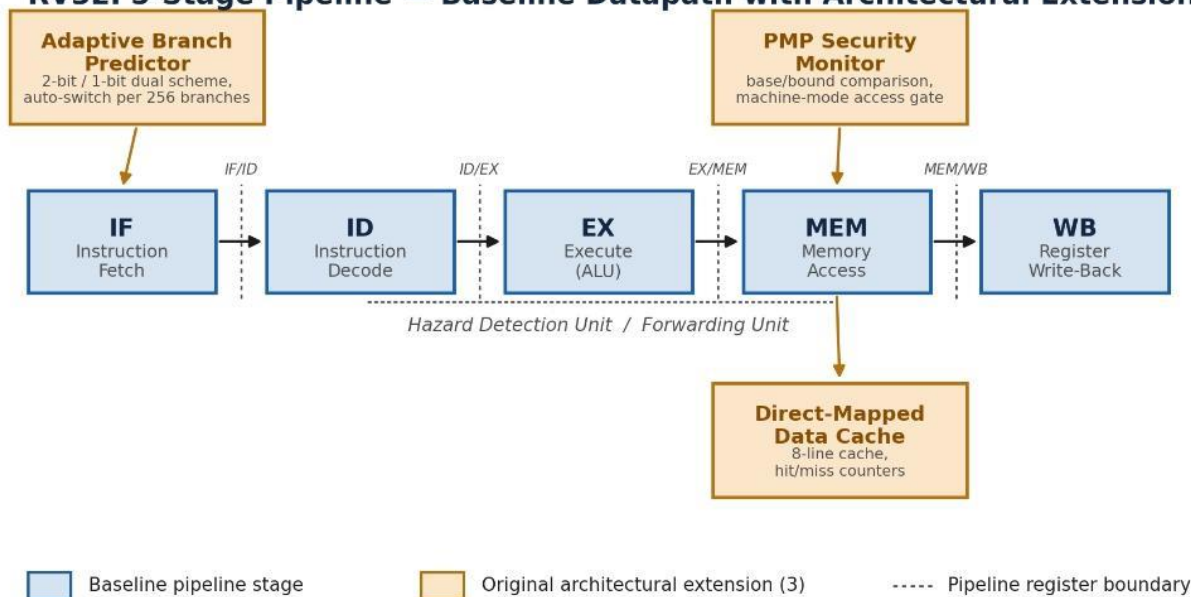
Tools used: draw.io (block diagrams)

What was done:

- Chose RV32I instruction set, 5-stage pipeline (IF - ID - EX - MEM - WB) as the baseline datapath.
- Scoped in three architectural extensions up front: adaptive branch predictor, PMP security monitor, direct-mapped data cache.

Output / Screenshot:

RV32I 5-Stage Pipeline — Baseline Datapath with Architectural Extensions



2. RTL Design (Verilog)

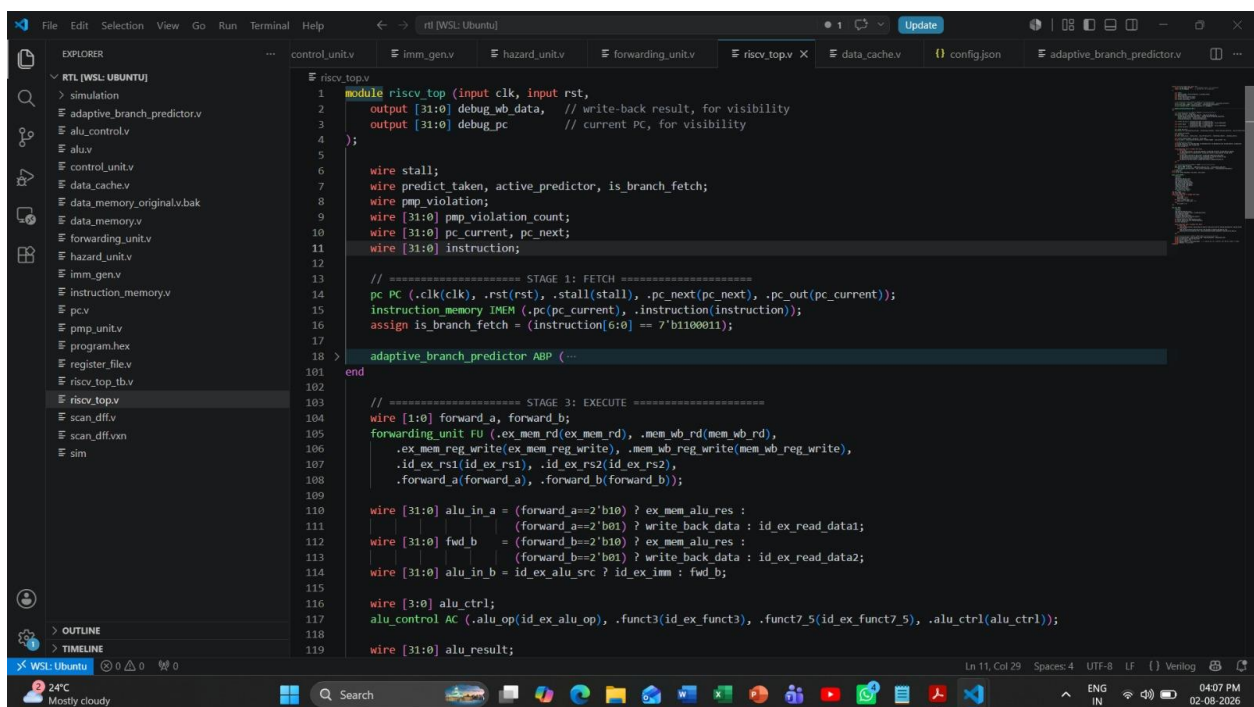
Objective: Implement the processor's core datapath and control logic in synthesizable Verilog.

Tools used: VS Code, Verilog-HDL/SystemVerilog extension

What was done:

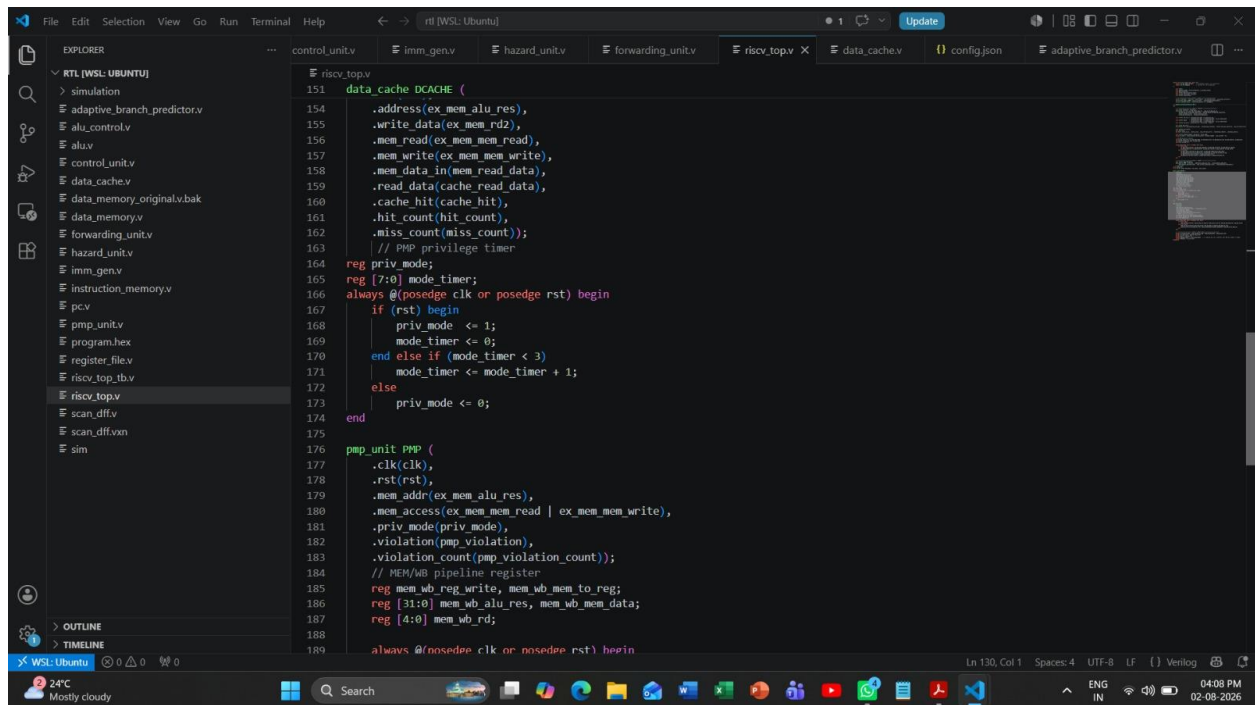
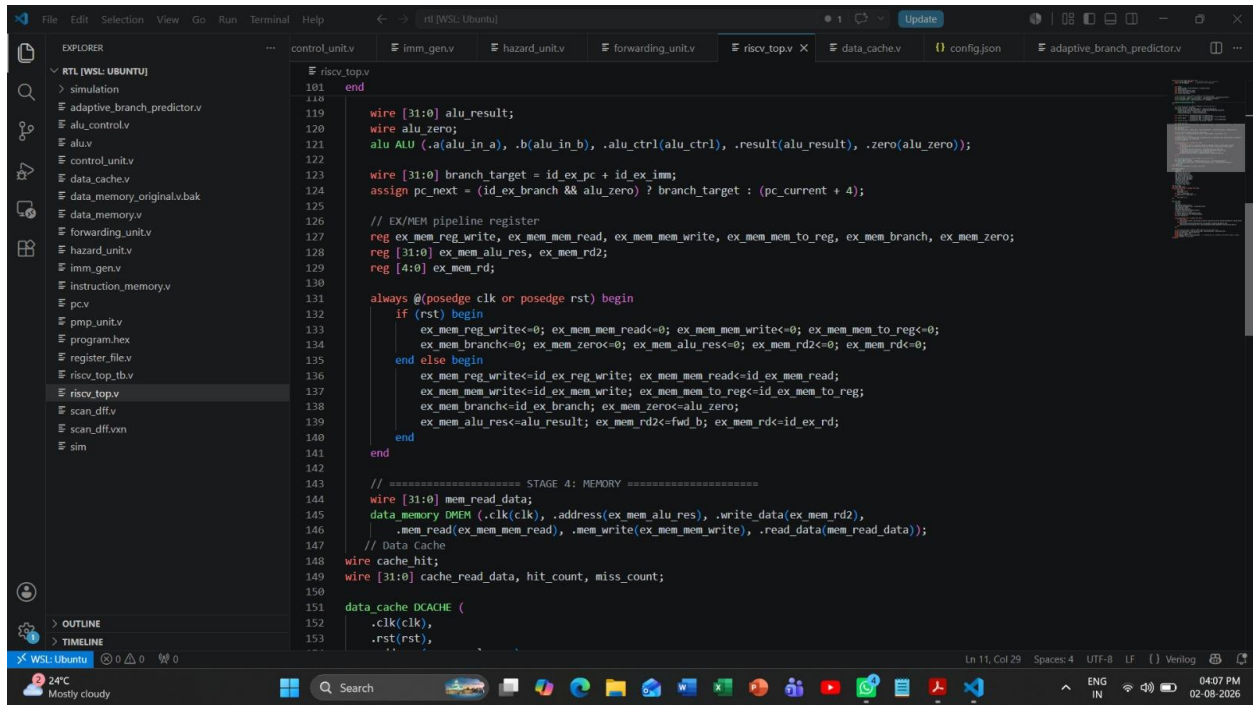
- Wrote 11 core modules: ALU, register file, instruction memory, data memory, program counter, immediate generator, control unit, ALU control, hazard detection unit, forwarding unit, and the top-level integration.
- Wired all modules together with IF/ID, ID/EX, EX/MEM and MEM/WB pipeline registers, including hazard stalling and forwarding.

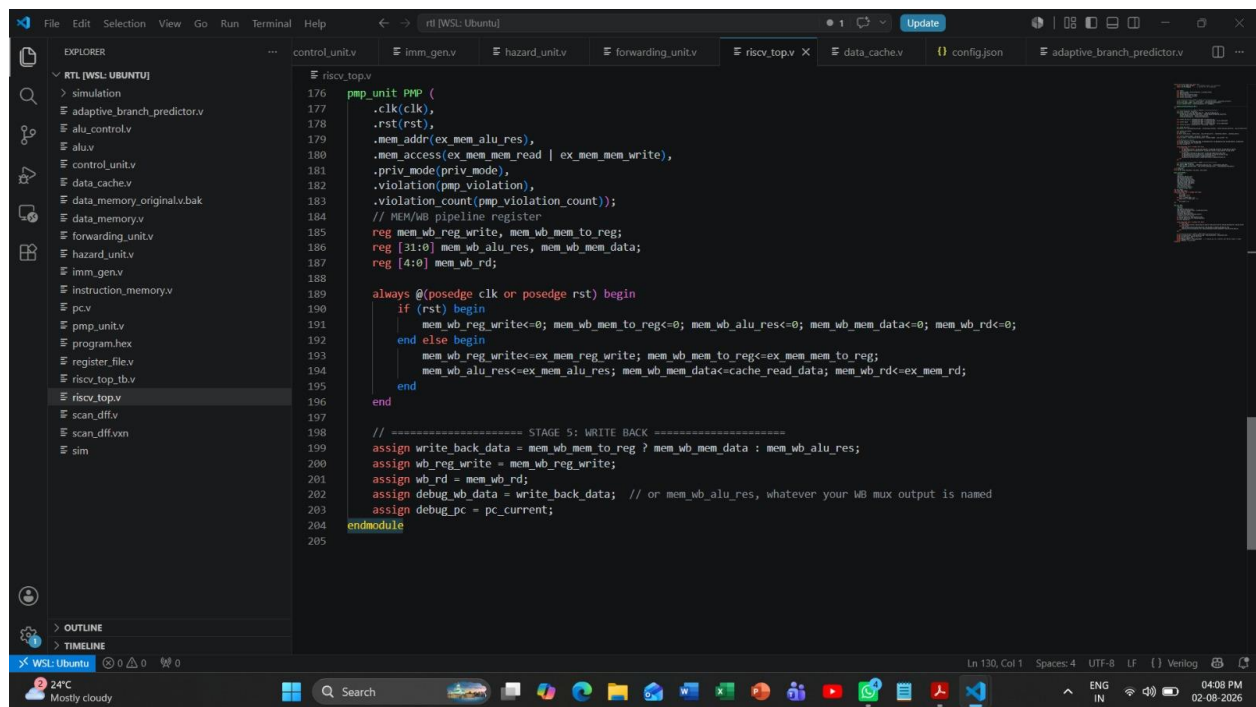
Output/ Screenshot:



The screenshot shows a Verilog code editor with the file explorer on the left displaying a project structure for 'RTL (WSL: UBUNTU)'. The main editor window shows the 'riscv_top.v' file, which is a top-level module for a RISC-V processor. The code defines inputs for clock (clk), reset (rst), and outputs for debug (debug_wb_data, debug_pc). It includes several internal signals and modules for pipeline stages (IF, ID, EX, MEM, WB), hazard detection, forwarding, and branch prediction. The code is written in Verilog and includes comments for various stages and components.

```
1 module riscv_top (input clk, input rst,
2   output [31:0] debug_wb_data, // write-back result, for visibility
3   output [31:0] debug_pc      // current PC, for visibility
4 );
5
6   wire stall;
7   wire predict_taken, active_predictor, is_branch_fetch;
8   wire pmp_violation;
9   wire [31:0] pmp_violation_count;
10  wire [31:0] pc_current, pc_next;
11  wire [31:0] instruction;
12
13  // ===== STAGE 1: FETCH =====
14  pc PC (.clk(clk), .rst(rst), .stall(stall), .pc_next(pc_next), .pc_out(pc_current));
15  instruction_memory IMEM (.pc(pc_current), .instruction(instruction));
16  assign is_branch_fetch = (instruction[6:0] == 7'b100011);
17
18  > adaptive_branch_predictor ABP (---
19
20  end
21
22  // ===== STAGE 3: EXECUTE =====
23  wire [1:0] forward_a, forward_b;
24  forwarding_unit FU (.ex_mem_rd(ex_mem_rd), .mem_wb_rd(mem_wb_rd),
25    .ex_mem_reg_write(ex_mem_reg_write), .mem_wb_reg_write(mem_wb_reg_write),
26    .id_ex_rs1(id_ex_rs1), .id_ex_rs2(id_ex_rs2),
27    .forward_a(forward_a), .forward_b(forward_b));
28
29  wire [31:0] alu_in_a = (forward_a==2'b10) ? ex_mem_alu_res :
30    (forward_a==2'b01) ? write_back_data : id_ex_read_data1;
31  wire [31:0] fwd_b = (forward_b==2'b10) ? ex_mem_alu_res :
32    (forward_b==2'b01) ? write_back_data : id_ex_read_data2;
33  wire [31:0] alu_in_b = id_ex_alu_src ? id_ex_imm : fwd_b;
34
35  wire [3:0] alu_ctrl;
36  alu_control AC (.alu_op(id_ex_alu_op), .funct3(id_ex_funct3), .funct7_5(id_ex_funct7_5), .alu_ctrl(alu_ctrl));
37
38  wire [31:0] alu_result;
```



3. Functional Simulation

Objective: Verify the baseline processor executes a test program correctly before adding any extensions.

Tools used: Icarus Verilog, GTKWave

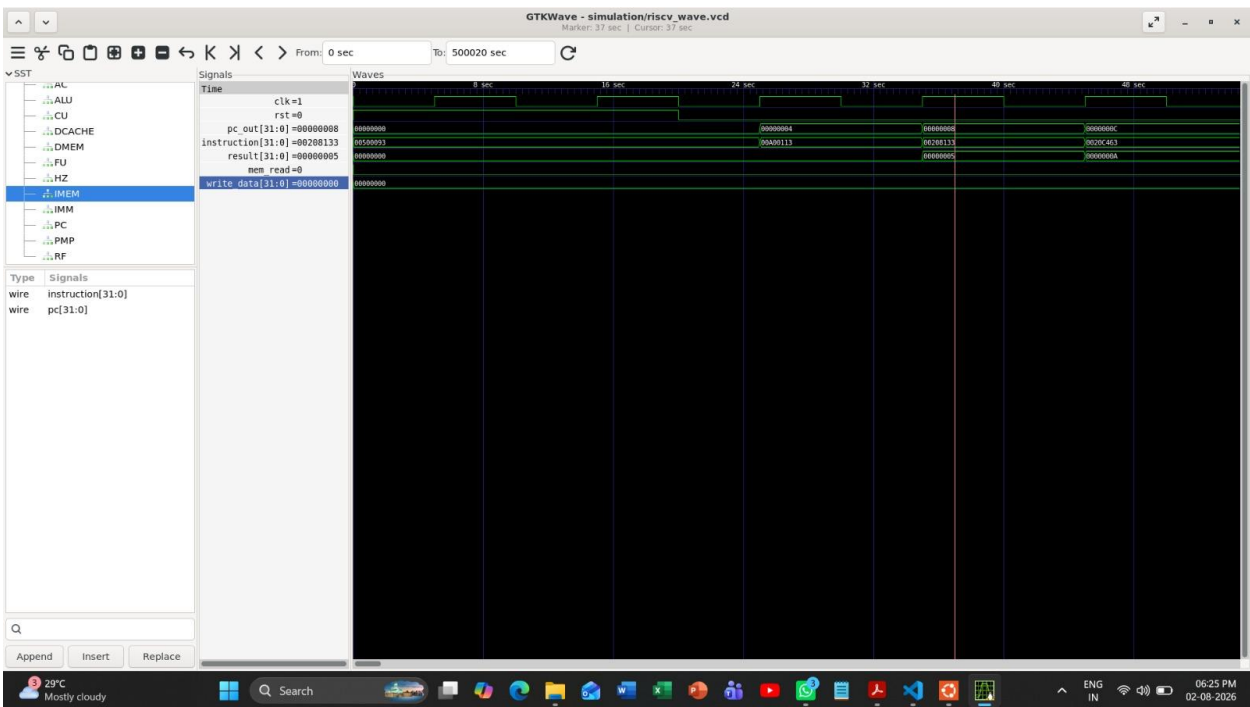
What was done:

- Wrote a top-level testbench driving clk/rst and dumping a VCD waveform.
- Ran a short test program (ADDI, ADD, SW, LW) and confirmed correct execution via GTKWave.

Output/ Screenshot:

```
vlsi@DESKTOP-TNE7RAE: ~/riscv_project
t=499745 pc=00030cd4 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499755 pc=00030cd8 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499765 pc=00030cdc instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499775 pc=00030ce0 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499785 pc=00030ce4 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499795 pc=00030ce8 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499805 pc=00030cec instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499815 pc=00030cf0 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499825 pc=00030cf4 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499835 pc=00030cf8 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499845 pc=00030cfc instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499855 pc=00030d00 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499865 pc=00030d04 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499875 pc=00030d08 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499885 pc=00030d0c instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499895 pc=00030d10 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499905 pc=00030d14 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499915 pc=00030d18 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499925 pc=00030d1c instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499935 pc=00030d20 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499945 pc=00030d24 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499955 pc=00030d28 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499965 pc=00030d2c instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499975 pc=00030d30 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499985 pc=00030d34 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=499995 pc=00030d38 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=500005 pc=00030d3c instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
t=500015 pc=00030d40 instr=xxxxxxxx id_ex_branch=0 alu_zero=x ex_mem_alu_res=xxxxxxxx ex_mem_rd2=xxxxxxxx wb=xxxxxxxx
Simulation complete at time 500020
testbench/riscv_tb.v:14: $finish called at 500020 (1s)
vlsi@DESKTOP-TNE7RAE:~/riscv_project$ wc -l program.hex
5 program.hex
vlsi@DESKTOP-TNE7RAE:~/riscv_project$ sed -n '1,15p' rtl/instruction_memory.v
module instruction_memory (
    input  [31:0] pc,
    output [31:0] instruction
);
    reg [31:0] mem [0:255];

    initial begin
        mem[0] = 32'h00500093; // ADDI x1,x0,5
        mem[1] = 32'h00A00113; // ADDI x2,x0,10
        mem[2] = 32'h00208133; // ADD x2,x1,x2 (-15)
        mem[3] = 32'h0020c463; // BLT x1,x2,+8 (taken)
        mem[4] = 32'h00100193; // ADDI x3,x0,1 (skipped)
        mem[5] = 32'h00312623; // SW x2,x(x0)
        mem[6] = 32'h00002283; // LW x5,0(x0)
    end
endmodule
```



4. Adaptive Branch Predictor

Objective: Add a novel branch-prediction module that runs two predictors in parallel and adapts to whichever performs better at runtime.

Tools used: Icarus Verilog, GTKWave

What was done:

- Built a module running a 2-bit and a 1-bit predictor in parallel, each indexed by the fetch PC.
- Tracked mispredictions per predictor and switched the 'active' predictor every 256 resolved branches.
- Wired the module into the fetch stage and extended the test program with branch instructions.

5. PMP Security Monitor

Objective: Add a security-monitoring module that flags unauthorized memory accesses.

Tools used: Icarus Verilog, GTKWave

What was done:

- Built a module that compares every memory address against a protected base/bound range.
- Flagged and counted any access made while not in machine (trusted) mode.
- Wired the module into the memory stage, alongside a simple mode timer to demonstrate the check.

6. Direct-Mapped Data Cache

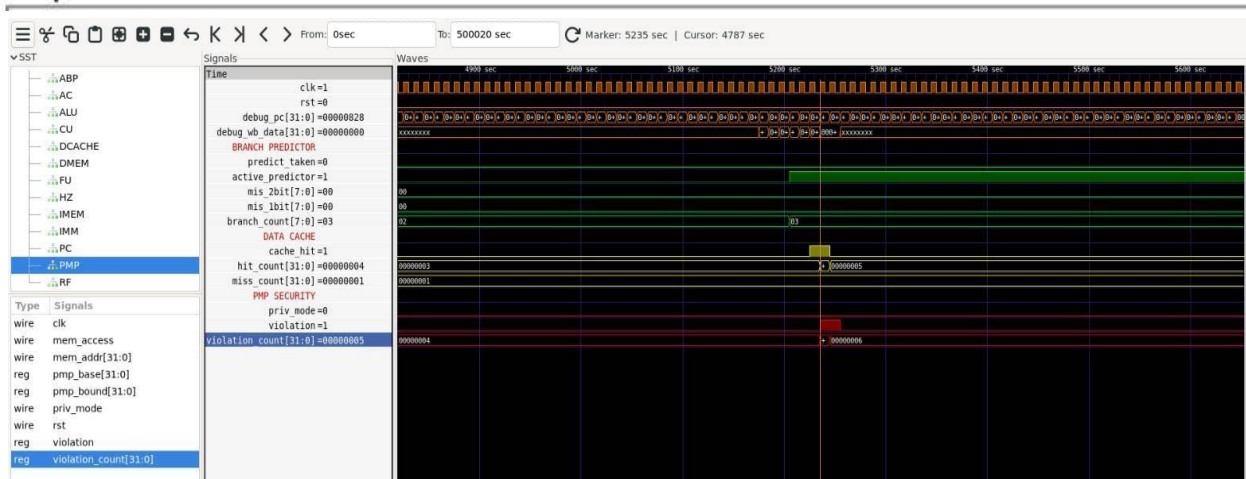
Objective: Add a small cache to speed up repeated data memory accesses.

Tools used: Icarus Verilog, GTKWave

What was done:

- Built an 8-line direct-mapped cache sitting between the CPU and data memory.
- Tracked hit/miss counts and served repeated loads from the cache instead of memory.
- Wired it in as a drop-in replacement ahead of the MEM/WB pipeline register.

Output / Screenshot:



7. Logic Synthesis

Objective: Convert the complete RTL (baseline + all three extensions) into a gate-level netlist and measure its size.

Tools used: Yosys

What was done:

- Re-ran Yosys synthesis on the full design, targeting riscv_top as the top module.
- Compared gate count, flip-flop count, and area against the original baseline-only synthesis to quantify the resource cost of the added features.

Output / Screenshot:

Logic Gate Count Table

Synthesized with Yosys 0.52 (synth -top riscv_top). Baseline = 11 core modules. Enhanced = baseline + adaptive_branch_predictor.v + pmp_unit.v + data_cache.v.

Gate / Cell Type	Baseline	Enhanced	Delta
\$_ANDNOT_ (AND-NOT gate)	3,516	4,017	+501
\$_AND_ (AND gate)	94	126	+32
\$_DFFE_PN_ (D flip-flop, enable)	0	256	+256
\$_DFFE_PPON_ (D-FF, enable, active-low)	32	32	0
\$_DFFE_PP0P_ (D-FF, enable, active-high)	1,056	1,128	+72
\$_DFFE_PP_ (D flip-flop, enable)	8,192	8,400	+208
\$_DFF_PP0_ (D-FF, async reset)	299	299	0
\$_MUX_ (2:1 multiplexer)	8,759	10,130	+1,371
\$_NAND_ (NAND gate)	124	155	+31
\$_NOR_ (NOR gate)	67	67	0
\$_NOT_ (inverter)	99	104	+5
\$_ORNOT_ (OR-NOT gate)	125	182	+57
\$_OR_ (OR gate)	2,481	2,805	+324
\$_XNOR_ (XNOR gate)	68	68	0
\$_XOR_ (XOR gate)	150	238	+88
TOTAL CELLS	25,062	28,007	+2,945 (+11.8%)

Flip-flops (all \$_DFF*_ types): 9,579 (baseline) → 10,115 (enhanced), +536 (+5.6%). Overall cell-count overhead for all three features together is ≈11.8%.

8. Gate-Level Verification

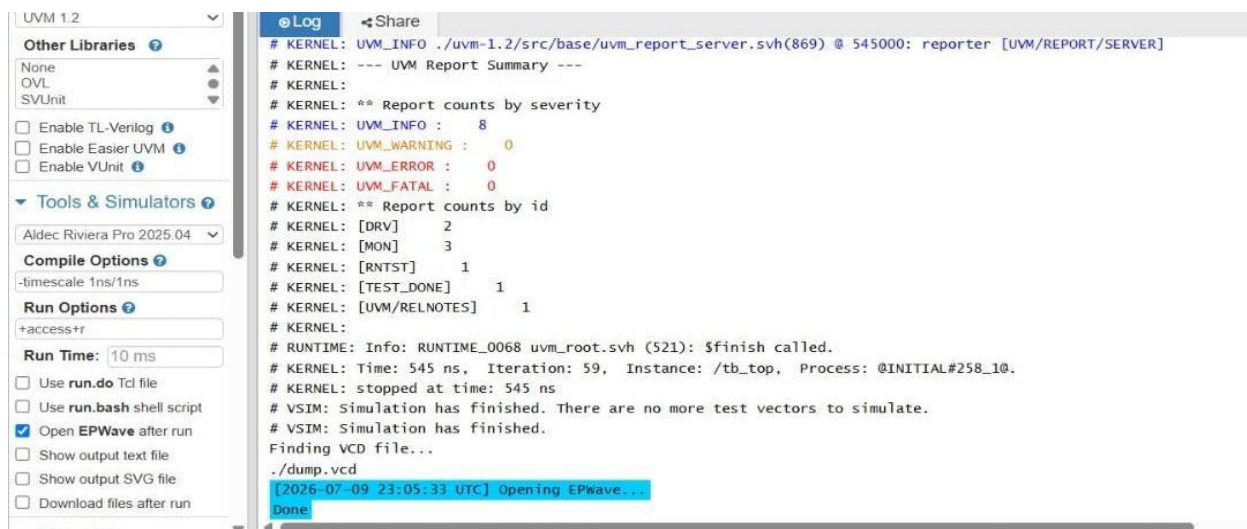
Objective: Verify the gate-level design automatically, rather than by manual waveform inspection.

Tools used: EDA Playground, Riviera-PRO, SystemVerilog/UVM

What was done:

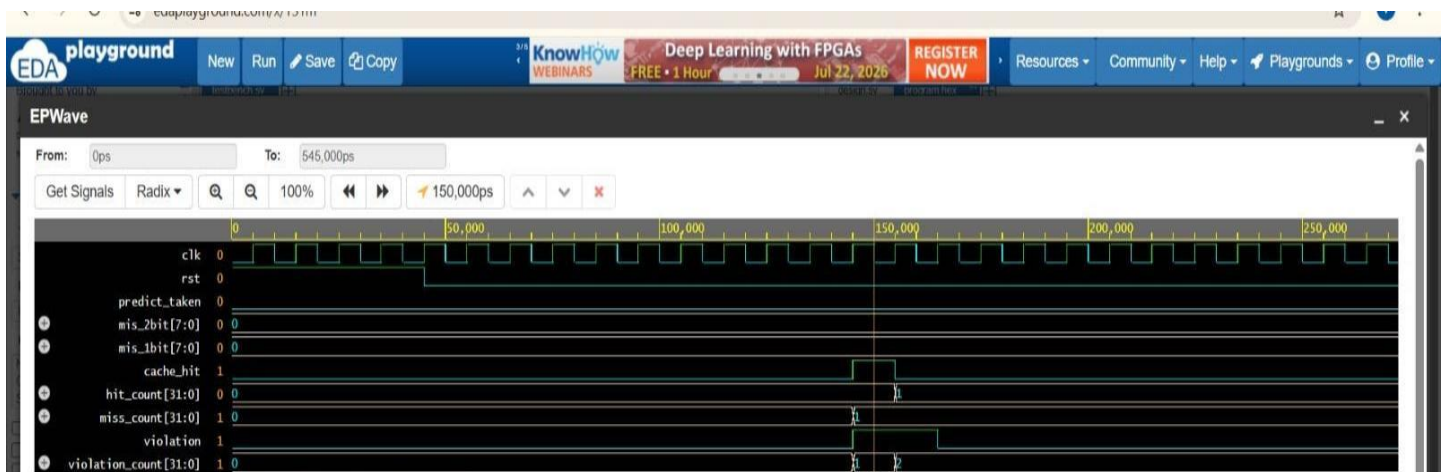
- Built a full UVM testbench: interface, transaction, sequence, driver, monitor, scoreboard, agent, environment, and test.
- Ran it on EDA Playground (Riviera-PRO), since Icarus Verilog does not support the UVM class library.
- Scoreboard automatically checked register values, branch-predictor activity, cache hit/miss counts, and PMP violation counts.

Output / Screenshot:



```
UVM 1.2
Other Libraries
None
OVL
SVUnit
[ ] Enable TL-Verilog
[ ] Enable Easier UVM
[ ] Enable VUnit
Tools & Simulators
Aldec Riviera Pro 2025.04
Compile Options
-timescale 1ns/1ns
Run Options
+access+tr
Run Time: 10 ms
[ ] Use run.do Tcl file
[ ] Use run.bash shell script
[ ] Open EPWave after run
[ ] Show output text file
[ ] Show output SVG file
[ ] Download files after run

Log
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_report_server.svh(869) @ 545000: reporter [UVM/REPORT/SERVER]
# KERNEL: --- UVM Report Summary ---
# KERNEL:
# KERNEL: ** Report counts by severity
# KERNEL: UVM_INFO : 8
# KERNEL: UVM_WARNING : 0
# KERNEL: UVM_ERROR : 0
# KERNEL: UVM_FATAL : 0
# KERNEL: ** Report counts by id
# KERNEL: [DRV] 2
# KERNEL: [MON] 3
# KERNEL: [RNTST] 1
# KERNEL: [TEST_DONE] 1
# KERNEL: [UVM/RELNOTES] 1
# KERNEL:
# KERNEL: Info: RUNTIME_0068 uvm_root.svh (521): $finish called.
# KERNEL: Time: 545 ns, Iteration: 59, Instance: /tb_top, Process: @INITIAL#258_10.
# KERNEL: stopped at time: 545 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2026-07-09 23:05:33 UTC] Opening EPWave...
Done
```



9. Functional Coverage

Objective: Measure how thoroughly the test scenarios exercised the design, not just whether it passed.

Tools used: EDA Playground, Riviera -PRO, SystemVerilog/UVM

What was done:

- Added covergroups to the UVM monitor for the branch predictor, cache, and PMP signals.
- Identified coverage gaps (e.g. a legal, non-violating PMP access never exercised) and extended the test program to close them.

Output/ Screenshot:

The screenshot shows the EDA Playground interface. On the left, there's a sidebar with 'Languages & Libraries' and 'Tools & Simulators'. The main area displays a SystemVerilog testbench file named 'testbench.sv'. The code includes a UVM testbench structure with a 'run' method that triggers a reset and then runs a test. The log window at the bottom shows the execution output, including UVM messages and functional coverage reports. The coverage report indicates that the test exercised various components, including the branch predictor, cache, and PMP signals, with an overall coverage of 80.56%.

```
169 if (tb_top.dut.RF.registers[5] == 32'd15) begin
170   uvm_info("SB", "CHECK 1 PASSED: x5 == 15 (LW after ADD executed correctly)",
171     UVM_NONE)
172   pass_count++;
173 end else begin
174   uvm_error("SB", $formatf("CHECK 1 FAILED: x5 = %0d, expected 15",
175     tb_top.dut.RF.registers[5]))
176   fail_count++;
177 end
```

Log output:

```
# KERNEL: See http://www.eda.org/svao/view.php?id=3//u for more details.
# KERNEL:
# KERNEL: (Specify +UVM_NO_RELNOTES to turn off this notice)
# KERNEL:
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: UVM_INFO @ 0: reporter [RNTST] Running test riscv_base_test...
# KERNEL: UVM_INFO /home/runner/testbench.sv(54) @ 0: uvm_test_top.env.agent.drv [DRV] Applying reset for 5 cycles, then running 45 cycles
# KERNEL: UVM_INFO /home/runner/testbench.sv(58) @ 45000: uvm_test_top.env.agent.drv [DRV] Reset released
# KERNEL: UVM_INFO /home/runner/testbench.sv(124) @ 155000: uvm_test_top.env.agent.mon [MON] [Cache] HIT hit_count=0 miss_count=1
# KERNEL: UVM_INFO /home/runner/testbench.sv(135) @ 155000: uvm_test_top.env.agent.mon [MON] [PMP] VIOLATION detected, count=1
# KERNEL: UVM_INFO /home/runner/testbench.sv(135) @ 165000: uvm_test_top.env.agent.mon [MON] [PMP] VIOLATION detected, count=2
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_objection.svh(1271) @ 545000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(143) @ 545000: uvm_test_top.env.agent.mon [COV] === FUNCTIONAL COVERAGE REPORT ===
# KERNEL: UVM_INFO /home/runner/testbench.sv(144) @ 545000: uvm_test_top.env.agent.mon [COV] cg_branch_predictor : 41.67 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(145) @ 545000: uvm_test_top.env.agent.mon [COV] - predict_taken_cp : 50.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(146) @ 545000: uvm_test_top.env.agent.mon [COV] - active_predictor_cp : 50.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(147) @ 545000: uvm_test_top.env.agent.mon [COV] - cross_pred_active : 25.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(148) @ 545000: uvm_test_top.env.agent.mon [COV] cg_cache : 100.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(149) @ 545000: uvm_test_top.env.agent.mon [COV] cg_pmp : 100.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(151) @ 545000: uvm_test_top.env.agent.mon [COV] === OVERALL COVERAGE: 80.56 % ===
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_report_server.svh(869) @ 545000: reporter [UVM/REPORT/SERVER]
# KERNEL: --- UVM Report Summary ---
# KERNEL:
```

```
# KERNEL: UVM_INFO /home/runner/testbench.sv(143) @ 545000: uvm_test_top.env.agent.mon [COV] === FUNCTIONAL COVERAGE REPORT ===
# KERNEL: UVM_INFO /home/runner/testbench.sv(144) @ 545000: uvm_test_top.env.agent.mon [COV] cg_branch_predictor : 41.67 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(145) @ 545000: uvm_test_top.env.agent.mon [COV] - predict_taken_cp : 50.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(146) @ 545000: uvm_test_top.env.agent.mon [COV] - active_predictor_cp : 50.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(147) @ 545000: uvm_test_top.env.agent.mon [COV] - cross_pred_active : 25.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(148) @ 545000: uvm_test_top.env.agent.mon [COV] cg_cache : 100.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(149) @ 545000: uvm_test_top.env.agent.mon [COV] cg_pmp : 100.00 %
# KERNEL: UVM_INFO /home/runner/testbench.sv(151) @ 545000: uvm_test_top.env.agent.mon [COV] === OVERALL COVERAGE: 80.56 % ===
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_report_server.svh(869) @ 545000: reporter [UVM/REPORT/SERVER]
# KERNEL: --- UVM Report Summary ---
# KERNEL:
```

10. Static Timing Analysis

Objective: Determine the processor's real maximum clock frequency (Fmax).

Tools used: OpenSTA, SkyWater Sky130 PDK

What was done:

- Re-synthesized the design mapped to real Sky130 standard cells (dfflibmap + abc).
- Wrote a clock constraint (SDC) and ran OpenSTA to find the critical path and setup/hold slack.
- Binary-searched the clock period to find the true minimum period and computed Fmax.

Output / Screenshot:

```
vlsi@DESKTOP-TNE7RAE: ~/riscv_project/sta
-1.74  4.16  library setup time
        4.16  data required time
-----
        4.16  data required time
-288.48  data arrival time
-----
-284.32  slack (VIOLATED)

Startpoint: _3553_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: DMEM/_26111_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

  Delay   Time   Description
-----
    0.00    0.00  clock clk (rise edge)
    0.00    0.00  clock network delay (ideal)
    0.00    0.00  ^ _3553_/CLK (sky130_fd_sc_hd_dfrtp_1)
   60.78   60.78  ^ _3553_/Q (sky130_fd_sc_hd_dfrtp_1)
  184.37  245.16  v DMEM/_21527_/X (sky130_fd_sc_hd_lpflow_isobufsrc_1)
   36.82  281.98  ^ DMEM/_21581_/Y (sky130_fd_sc_hd_nand3_1)
    6.50  288.48  v DMEM/_21817_/Y (sky130_fd_sc_hd_nor2_1)
    0.00  288.48  v DMEM/_26111_/DE (sky130_fd_sc_hd_edfxt_1)
   288.48  data arrival time

    6.00    6.00  clock clk (rise edge)
    0.00    6.00  clock network delay (ideal)
   -0.10    5.90  clock uncertainty
    0.00    5.90  clock reconvergence pessimism
    5.90    5.90  ^ DMEM/_26111_/CLK (sky130_fd_sc_hd_edfxt_1)
   -1.74    4.16  library setup time
        4.16  data required time
-----
        4.16  data required time
-288.48  data arrival time
-----
-284.32  slack (VIOLATED)

Saving command history
=== WORST NEGATIVE SLACK (WNS) ===
wns -284.32

=== TOTAL NEGATIVE SLACK (TNS) ===
tns -1646265.38

=== TOP 5 CRITICAL PATHS ===
vlsi@DESKTOP-TNE7RAE:~/riscv_project/sta$
```


11. DFT (Scan Chain Insertion)

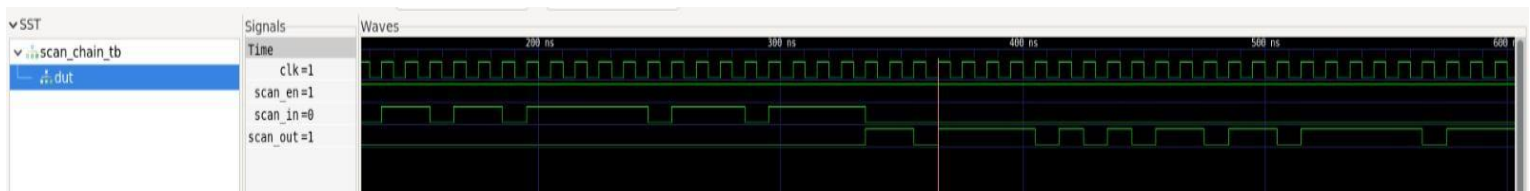
Objective: Add design-for-test structures so the manufactured chip's internal state can be tested post-fabrication.

Tools used: Icarus Verilog, GTKWave

What was done:

- Replaced pipeline flip-flops with scan flip-flops, chaining scan_out of one stage to scan_in of the next.
- Built a scan-chain testbench that shifts a test pattern through the full chain and checks it comes back out unchanged.

Output/ Screenshot:



```
iverilog -o scan_tb rtl/scan_dff.v tb/scan_chain_tb.v
vvp scan_tb > scan_debug.log
tail -20 scan_debug.log
Patched.
t=485000 captured[17]=0 q=6fbbc000
t=495000 captured[16]=1 q=df778000
t=505000 captured[15]=1 q=beef0000
t=515000 captured[14]=0 q=7dde0000
t=525000 captured[13]=1 q=fbbc0000
t=535000 captured[12]=1 q=f7780000
t=545000 captured[11]=1 q=eef00000
t=555000 captured[10]=1 q=dde00000
t=565000 captured[9]=1 q=bbc00000
t=575000 captured[8]=0 q=77800000
t=585000 captured[7]=1 q=ef000000
t=595000 captured[6]=1 q=de000000
t=605000 captured[5]=1 q=bc000000
t=615000 captured[4]=0 q=78000000
t=625000 captured[3]=1 q=f0000000
t=635000 captured[2]=1 q=e0000000
t=645000 captured[1]=1 q=c0000000
t=655000 captured[0]=1 q=80000000
Scan chain integrity : PASS (32/32 bits match)
tb/scan_chain_tb.v:54: $finish called at 655000 (1ps)
vlsi@DESKTOP-TNE7RAE:~/riscv_project$
```

12. Physical Design

Objective: Turn the gate-level netlist (with scan chain) into a physical chip layout.

Tools used: OpenLane2, OpenROAD, KLayout, Sky130 PDK

What was done:

- Ran the OpenLane2 interactive flow: synthesis, floorplan, placement, clock tree synthesis, routing, parasitics extraction, and post-route STA.
- Reviewed the routed layout in KLayout and the numeric summary report (cell count, wirelength, WNS, DRC count).

Output/ Screenshot:

```
vs1@DESKTOP-TNE7BAE: ~/OpenLane
[WARNING]: There are max fanout violations in the design at the typical corner. Please refer to 'designs/spw/runs/openlane_test/reports/signoff/31-rcx_sta.checks.rpt'.
[INFO]: There are no hold violations in the design at the typical corner.
[WARNING]: There are max fanout violations in the design at the typical corner.
[SUCCESS]: Flow complete.
[INFO]: Note that the following warnings have been generated:
[WARNING]: 1 warnings found by linter
[WARNING]: Current core area is too small for the power grid settings chosen. The power grid will be scaled down.
[WARNING]: Module sky130_fd_sc_hd_tapwrvvgnd_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd_decap_22 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd_fill_1 blackboxed during sta
[WARNING]: Module sky130_fd_sc_hd_fill_2 blackboxed during sta
[WARNING]: VSRC_LOC_FILES is not defined. The IR drop analysis will run, but the values may be inaccurate.
[WARNING]: There are max fanout violations in the design at the typical corner. Please refer to 'designs/spw/runs/openlane_test/reports/signoff/31-rcx_sta.checks.rpt'.

Basic test passed
vs1@DESKTOP-TNE7BAE:~/OpenLane$ cd ~/OpenLane
vs1@DESKTOP-TNE7BAE:~/OpenLane$ cd ~/riscv_project/src
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ mkdir -p designs/rv32i_top/src
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ cp -r project_backup/rtl/*.v designs/rv32i_top/src/
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ cp designs/rv32i_top/src/* ./
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ adaptive_branch_predictor.v alu_control.v data_cache.v forwarding_unit.v imm_gen.v pc.v register_file.v riscv_top_tb.v
alu.v control_unit.v data_memory.v hazard_unit.v instruction_memory.v pmp_unit.v riscv_top.v scan_dff.v
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ find ./project_backup -name "riscv_top.v"
./project_backup/rtl/riscv_top.v
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ cp ./project_backup/OpenLane_designs/src/riscv_top.v
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ cp ./riscv_top.v
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ cp -r project_backup/OpenLane_designs/config.json designs/rv32i_top/config.json
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$ cat designs/rv32i_top/config.json
{
  "DESIGN_NAME": "riscv_top",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 20,
  "FP_SIZING": "absolute",
  "DIE_AREA": "0 0 1000 1000",
  "PL_TARGET_DENSITY": 0.55,
  "SYNTH_STRATEGY": "AREA 0",
  "PDK": "sky130A"
}
vs1@DESKTOP-TNE7BAE:~/riscv_project/src$
```

```
"DESIGN_NAME": "riscv_top",
"VERILOG_FILES": "dir::src/*.v",
"CLOCK_PORT": "clk",
"CLOCK_PERIOD": 20,
"FP_SIZING": "absolute",
"DIE_AREA": "0 0 1000 1000",
"PL_TARGET_DENSITY": 0.55,
"SYNTH_STRATEGY": "AREA 0",
"PDK": "sky130A"
```

```

[STEP 1]
[INFO]: Running Synthesis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.43.25/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.43.25/logs/synthesis/2-sta.log)...
% ^C
OpenLane Container (ff5509f):/openlane$ ./flow.tcl -interactive
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.
Available under the Apache License, version 2.0. See the LICENSE file for more details.

% package require openlane
1.0.2
% prep -design rv32i_top
[INFO]: Using configuration in 'designs/rv32i_top/config.json'...
[INFO]: PDK Root: /home/geeth/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/rv32i_top/runs/RUN_2026.07.29_20.53.16
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
% run_synthesis
[STEP 1]
[INFO]: Running Synthesis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/synthesis/2-sta.log)...
% run_floorplan
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/floorplan/3-initial_fp.log)...
[INFO]: Floorplanned with width 988.54 and height 976.48.
[STEP 4]
[INFO]: Running IO Placement...
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/floorplan/5-tap.log)...
[INFO]: Power planning with power {VPWR} and ground {VGND}...
[STEP 6]
[INFO]: Generating PDN (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/floorplan/6-pdn.log)...

```

```

[STEP 8]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/placement/8-gpl_sta.log)...
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/placement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/placement/10-detailed.log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/placement/11-dpl_sta.log)...
% run_cts
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/cts/12-cts.log)...
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/cts/13-cts_sta.log)...
% run_routing
[STEP 14]
[INFO]: Running Global Routing Resizer Design Optimizations (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/14-resizer_design.log)...
[STEP 15]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/15-rsz_design_sta.log)...
[STEP 16]
[INFO]: Running Global Routing Resizer Timing Optimizations (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/16-resizer_timing.log)...
[STEP 17]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/17-rsz_timing_sta.log)...
[STEP 18]
[INFO]: Running Global Routing (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/18-global.log)...
[INFO]: Starting OpenROAD Antenna Repair Iterations...
[INFO]: Starting antenna repair iteration 1 with 1 violations...
[INFO]: [Iteration 1] Failed to reduce antenna violations (1 -> 1), stopping iterations...
[STEP 19]
[INFO]: Writing Verilog (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/18-global_write_netlist.log)...
[STEP 20]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/20-grt_sta.log)...
[STEP 21]
[INFO]: Running Fill Insertion (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/21-fill.log)...
[STEP 22]
[INFO]: Running Detailed Routing (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/22-detailed.log)...
[INFO]: No DRC violations after detailed routing.
[STEP 23]
[INFO]: Checking Wire Lengths (log: designs/rv32i_top/runs/RUN_2026.07.29_20.53.16/logs/routing/23-wire_lengths.log)...
1785361806
%

```


13. Physical Verification

Objective: Confirm the physical layout is manufacturable and matches the intended circuit.

Tools used: Magic, Netgen

What was done:

- Ran Design Rule Checking (DRC) in Magic and confirmed zero violations.
- Ran Layout-vs-Schematic (LVS) in Netgen and confirmed the layout matches the gate-level netlist.

Output/ Screenshot:

```
% run_magic
[STEP 25]
[INFO]: Running Magic to generate various views...
[INFO]: Streaming out GDSII with Magic (log: designs/rv32i_top/runs/drc_lvs_run4/logs/signoff/25-gdsii.log)...
[INFO]: Generating MAGLEF views...
[INFO]: Generating lef with Magic (/openlane/designs/rv32i_top/runs/drc_lvs_run4/logs/signoff/25-lef.log)...
% run_magic_drc
[STEP 26]
[INFO]: Running Magic DRC (log: designs/rv32i_top/runs/drc_lvs_run4/logs/signoff/26-drc.log)...
[INFO]: Converting Magic DRC database to various tool-readable formats...
[INFO]: No DRC violations after GDS streaming out.
%
```

```
OpenLane Container (ff5509f):/openlane$ cat designs/rv32i_top/runs/drc_lvs_run4/reports/signoff/33-riscv_top.lvs.rpt
LVS reports no net, device, pin, or property mismatches.

Total errors = 0
OpenLane Container (ff5509f):/openlane$ grep -i "match" designs/rv32i_top/runs/drc_lvs_run4/reports/signoff/33-riscv_top.lvs.rpt
LVS reports no net, device, pin, or property mismatches.
OpenLane Container (ff5509f):/openlane$ cat designs/rv32i_top/runs/drc_lvs_run4/reports/signoff/33-riscv_top.lvs.rpt
LVS reports no net, device, pin, or property mismatches.

Total errors = 0
```


14. GDSII (Tapeout-Ready Output)

Objective: Produce and archive the final manufacturable layout file.

Tools used: KLayout

What was done:

- Located the final GDSII file generated during physical design.
- Archived the GDS file alongside the DRC and LVS reports as the tapeout-ready deliverable.

Output/ Screenshot:

